# Product Requirements Document

High-Conversion Software Engineering Portfolio

`VERSION 1.0 // ARCHITECTURE & DESIGN SPECIFICATION`

## 1. Executive Summary & Strategy

This document outlines the architecture, design, and structural requirements for a high-performance software engineering portfolio. The objective is to convert profile views into technical interviews by prioritizing system design capabilities, problem-solving, and clean code presentation over unnecessary UI gimmicks.

> **Core Philosophy: Zero-Maintenance & Deployment-Friendly**
>
> The portfolio must act as a zero-maintenance asset. It relies on a static build process (SSG) with continuous integration. Updating the portfolio should require only pushing a Markdown/MDX file to a Git repository, with automated deployment to edge networks.

## 2. Technical Stack Definition

The chosen stack ensures a perfect Lighthouse score (SEO and Performance), rapid development, responsive orientation, and zero hosting costs.

| Layer | Technology | Rationale & Implementation Note |
| --- | --- | --- |
| **Framework** | `Next.js (App Router)` | Industry standard. Use React Server Components (RSC) for static generation. Ensures maximum SEO and frontend competence signaling. |
| **Styling** | `Tailwind CSS` | Utility-first CSS allows for rapid, consistent styling. Ensures deep responsiveness across all device orientations (portrait/landscape/tablet). |
| **Content Layer** | `MDX / local JSON` | Case studies and project data stored locally. No database dependency. Eliminates cold-start issues and backend vulnerabilities. |
| **Deployment** | `Vercel` or `GH Pages` | Automated CI/CD. The build step should be configuration-free. Vercel analytics can be injected safely. |

### Deployment & Orientation Friendly Requirements

- **Orientation Agnostic:** UI must use CSS Grid/Flexbox fluidly. It must degrade gracefully from ultra-

## 3. Design System & Theme: "Terminal Slate"

The visual design communicates engineering rigor. A "Dark Mode" aesthetic is requested as it aligns with developer IDEs, reduces eye strain, and provides high contrast for typography and code snippets.

| Token / Role | Hex Value | Application Context |
|---|---|---|
| **Background** | `#0f172a` | Main application background (Tailwind: bg-slate-900). |
| **Surface (Cards)** | `#1e293b` | Project cards, modal backgrounds, navbar (bg-slate-800). |
| **Primary Accent** | `#38bdf8` | Primary buttons, hover states, active links (text-sky-400). |
| **Secondary Accent** | `#10b981` | Success metrics, "Live" indicators, tech tags (text-emerald-500). |
| **Primary Text** | `#f8fafc` | Main headings and highly readable body text (text-slate-50). |
| **Muted Text** | `#94a3b8` | Subtitles, dates, secondary body text (text-slate-400). |

**Typography:** Use **Inter** (or system sans-serif) for body text and **Fira Code / JetBrains Mono** for code snippets, metrics, and labels.

## 4. Information Architecture & Sections

The layout follows a single-page or seamlessly hybrid flow. Content is structured to answer recruiter queries chronologically.

### Section 1: The Hero (Above the Fold)

- **Headline:** Clear, specific role definition (e.g., "Software Engineer specializing in Backend Systems & Cloud").
- **Sub-headline:** A concise value proposition focusing on outcomes.
- **Call to Action (CTA):** Primary button to "Featured Projects", secondary outline button to "Download Resume".
- **Social Proof:** Minimalist GitHub, LinkedIn, and Email icon links.

### Section 2: Technical Arsenal (Skills)

- **Design Pattern:** Avoid subjective percentage progress bars. Use categorized pill/badge tags.
- **Categories:** Backend & APIs, Cloud & DevOps, Frontend, Data/Architecture.
- **Mobile View:** Tags should wrap fluidly using `flex-wrap` with consistent gap spacing.

# 4. Information Architecture (Continued)

### Section 3: Featured Engineering Projects (The Core)

This is the most critical section. Limit to top 2-3 projects to avoid decision fatigue. Focus on system design.

- **Card Layout:** Grid layout (1 column on mobile, 2 columns on desktop).
- **Title & Context:** Project name and a 1-2 sentence description of the business/technical value.
- **Architecture Tags:** Visual pill tags of the specific stack used (e.g., `Docker`, `Redis`, `Go`).
- **"The Hard Part":** A unique prose block detailing a specific engineering challenge solved (e.g., "Reduced query latency by 40% through implementing a Redis caching layer for the primary dataset.").
- **Action Links:** "View Source (GitHub)" and "Live Demo / Case Study".

### Section 4: Experience & Education

- **Design Pattern:** Clean vertical timeline (using left borders and absolute positioned indicator dots).
- **Experience:** Detail internships, freelance, or commercial exposure. Focus on impact metrics over daily duties.
- **Education:** BSc Software Engineering (University Name). Highlight relevant advanced coursework (e.g., Distributed Systems, Machine Learning).

### Section 5: The Terminal (Contact/Footer)

- **Design Pattern:** A minimalist footer styled subtly like a command-line prompt.
- **Content:** `> connect --email user@example.com`. Direct mailto link, copyright, and final links.

# 5. Implementation Guidelines for AI Agents

When executing this PRD, the code agent must adhere to the following strict rules:

- Use semantic HTML5 tags (`<header>`, `<section>`, `<main>`, `<footer>`).
- Ensure WCAG accessibility (all images/icons must have `alt` tags or `aria-labels`, proper contrast ratios).
- Keep the component structure modular (e.g., separate `Hero.tsx`, `ProjectCard.tsx`, `Timeline.tsx`).
- Data for projects and experience should be extracted into a separate configuration file (e.g., `data/portfolio.ts`) to keep React components clean and easily modifiable.